

ARQUITECTURA DE DATOS

PROYECTO CAR SEATS

Lambda, Lakehouse y Pipeline Híbrido Modular

Asignatura: SCY1101 - Programación para la Ciencia de Datos

Docente: Mariela Moraga

Estudiante: Milenka Guerra

Evaluación Parcial N°3 - Documento técnico

Junio 2026

1. Introducción

Este documento analiza tres arquitecturas de datos usadas comúnmente en proyectos de ciencia de datos: Lambda, Lakehouse y Pipeline Híbrido Modular. El objetivo es determinar cuál es la más adecuada para el proyecto Car Seats que integra un dataset estático (CSV) y una API REST de tipo de cambio, y documentar, con su justificación y diagrama, la arquitectura efectivamente utilizada.

¿Qué es una arquitectura de datos?

Una arquitectura de datos define cómo fluye la información desde sus fuentes hasta los sistemas que la consumen. Determina cómo se extraen, almacenan, transforman y sirven los datos, y qué herramientas e infraestructura se utilizan en cada etapa.

Elegir la arquitectura correcta es fundamental, ya que una arquitectura sobredimensionada introduce complejidad innecesaria, y por otra parte, una subdimensionada no escala con el problema.

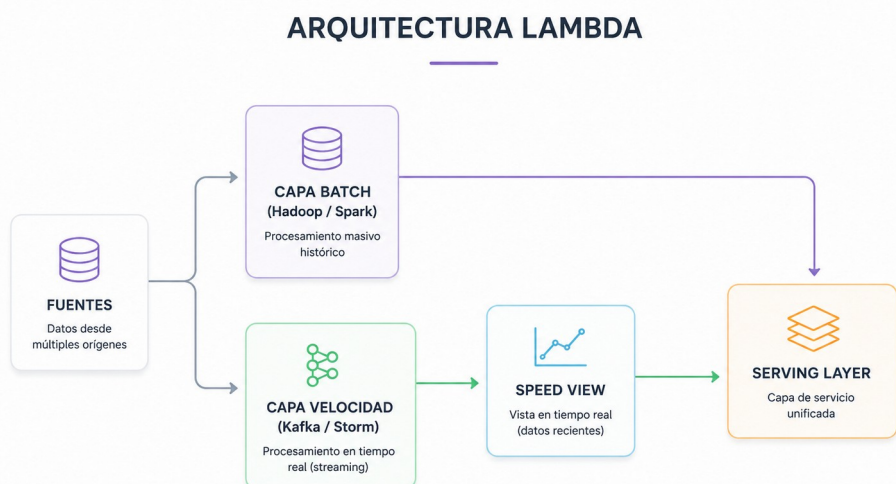
2. Descripción de las Arquitecturas

2.1 Arquitectura Lambda

Diseñada originalmente por Nathan Marz para sistemas de big data que necesitan procesar datos simultáneamente en tiempo real y en lotes. Combina tres capas paralelas:

- **Capa de velocidad (speed layer):** procesa eventos a medida que llegan y entrega resultados aproximados de inmediato.
- **Capa batch (batch layer):** procesa todo el histórico y entrega resultados exactos con mayor latencia.
- **Capa de servicio (serving layer):** combina ambas vistas y las expone para consultas.

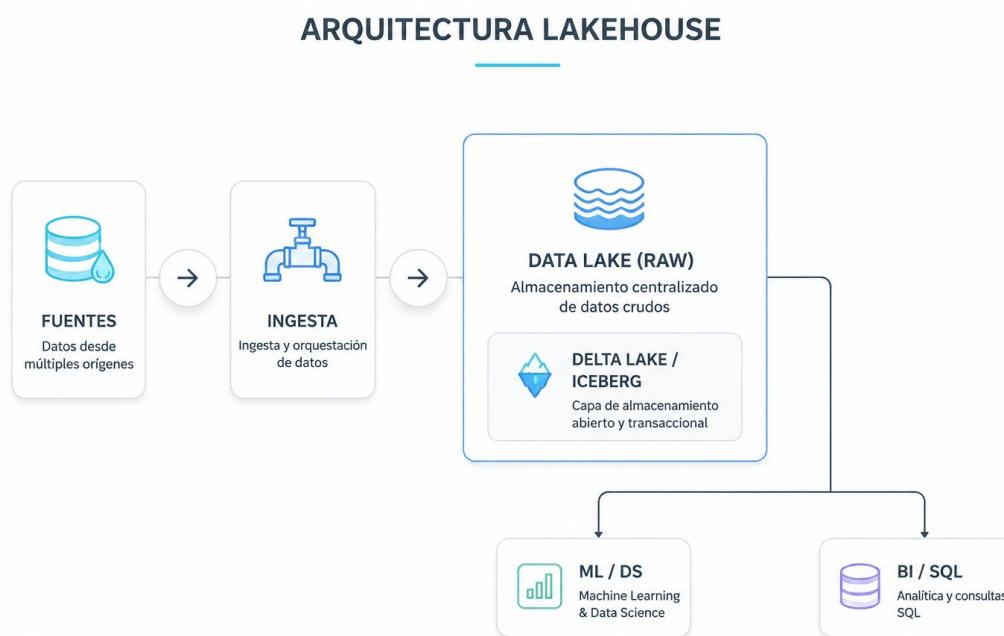
Se usa típicamente en detección de fraude, IoT o analítica de clickstream, con herramientas como Kafka o Spark Streaming para la capa de velocidad, y Hadoop o Spark para la capa batch.



2.2 Arquitectura Lakehouse

Unifica un **data lake** (almacenamiento económico y escalable de datos crudos, en cualquier formato) con las capacidades propias de un **data warehouse** (transacciones ACID, esquemas definidos, soporte SQL/BI), evitando mantener dos sistemas separados con datos duplicados.

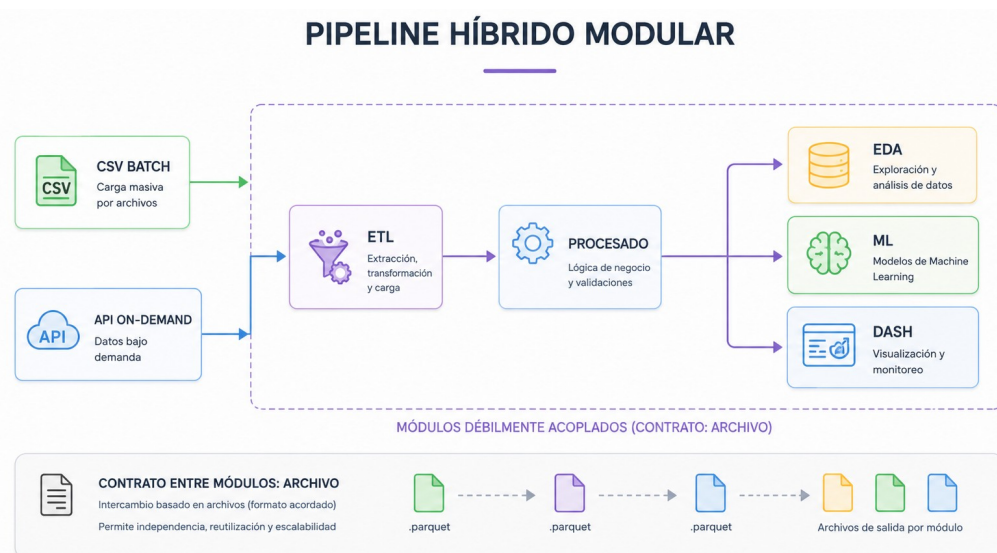
Ejemplos conocidos son Databricks (Delta Lake) y Apache Iceberg. Está pensada para organizaciones con múltiples fuentes de datos heterogéneas y a gran escala, que necesitan una sola plataforma para ciencia de datos, machine learning y reporting tradicional.



2.3 Pipeline Híbrido Modular

A diferencia de Lambda y Lakehouse, arquitecturas con nombre propio y un conjunto de herramientas específicas asociado, un Pipeline Híbrido Modular es un patrón de diseño más flexible. El flujo de datos se organiza en módulos independientes y débilmente acoplados (extracción, transformación, carga, consumo), que se comunican mediante contratos simples.

Es "híbrido" porque puede combinar fuentes de distinta naturaleza, como datos en lote (un CSV) y datos on-demand (una llamada a una API), sin necesitar la infraestructura de streaming de Lambda ni el motor de almacenamiento unificado de Lakehouse.



3. Comparación de Arquitecturas

La siguiente tabla resume las diferencias clave entre las tres arquitecturas evaluadas, considerando los criterios más relevantes para el proyecto Car Seats:

Criterio	Lambda	Lakehouse	Pipeline Híbrido Modular
Pensada para	Big data en tiempo real + batch	Múltiples fuentes a gran escala, BI + ML unificado	Proyectos con pocas fuentes heterogéneas, batch + on-demand
Complejidad de infraestructura	Alta (dos pipelines paralelos)	Alta (motor de almacenamiento especializado)	Baja a media (módulos simples)
Volumen de datos típico	Masivo (streaming continuo)	Masivo (múltiples TB, varias fuentes)	Pequeño a mediano
¿Necesita tiempo real?	Sí, es el problema que resuelve	No necesariamente	No, solo consultas on-demand puntuales
Costo / esfuerzo de implementación	Alto	Alto	Bajo
¿Adecuada para este proyecto?	✗ No	✗ No	✓ Sí

Conclusión de la comparación

Las tres arquitecturas tienen sus casos de uso legítimos. La clave no es elegir la más sofisticada, sino la que mejor se ajusta a la escala y naturaleza del problema:

- Lambda → streams masivos y tiempo real (fraude, IoT, clickstream).
- Lakehouse → organizaciones con decenas de fuentes y volúmenes de terabytes.
- Pipeline Híbrido Modular → proyectos académicos o medianos con pocas fuentes.

4. Justificación de la Arquitectura Elegida

4.1 Por qué no Arquitectura Lambda

El proyecto Car Seats integra exactamente dos fuentes de datos: un archivo CSV estático (400 filas, sin actualizaciones) y una API REST de tipo de cambio, consultada una vez por ejecución, con manejo de fallback en cadena. No existe un flujo continuo de datos en tiempo real, por lo que la Arquitectura Lambda no aplica y su complejidad (dos pipelines paralelos, infraestructura de streaming) no tiene ningún problema real que resolver aquí.

Razón de descarte: Lambda

Lambda fue diseñada para el problema de combinar resultados aproximados inmediatos con resultados exactos eventuales sobre streams masivos. En este proyecto:

- No hay stream de datos: el CSV tiene 400 filas estáticas.
- No se necesitan resultados en tiempo real.
- Implementar Kafka + Spark solo para leer un CSV sería sobreingeniería pura.

4.2 Por qué no Arquitectura Lakehouse

Tampoco se justifica una Arquitectura Lakehouse, ya que está pensada para organizaciones con decenas de fuentes heterogéneas y volúmenes de datos del orden de terabytes, que necesitan un motor de almacenamiento unificado para servir tanto a equipos de BI como de ciencia de datos. Implementarla para un proyecto de 400 filas y dos fuentes sería una sobreingeniería desproporcionada respecto al problema real.

Razón de descarte: Lakehouse

El Lakehouse resuelve la duplicación de datos entre un data lake y un data warehouse. En este proyecto:

- Solo hay dos fuentes de datos (CSV + API).
- No se requiere soporte SQL/BI empresarial ni transacciones ACID.
- Delta Lake o Iceberg añadirían infraestructura sin ningún beneficio real.

4.3 Por qué Pipeline Híbrido Modular

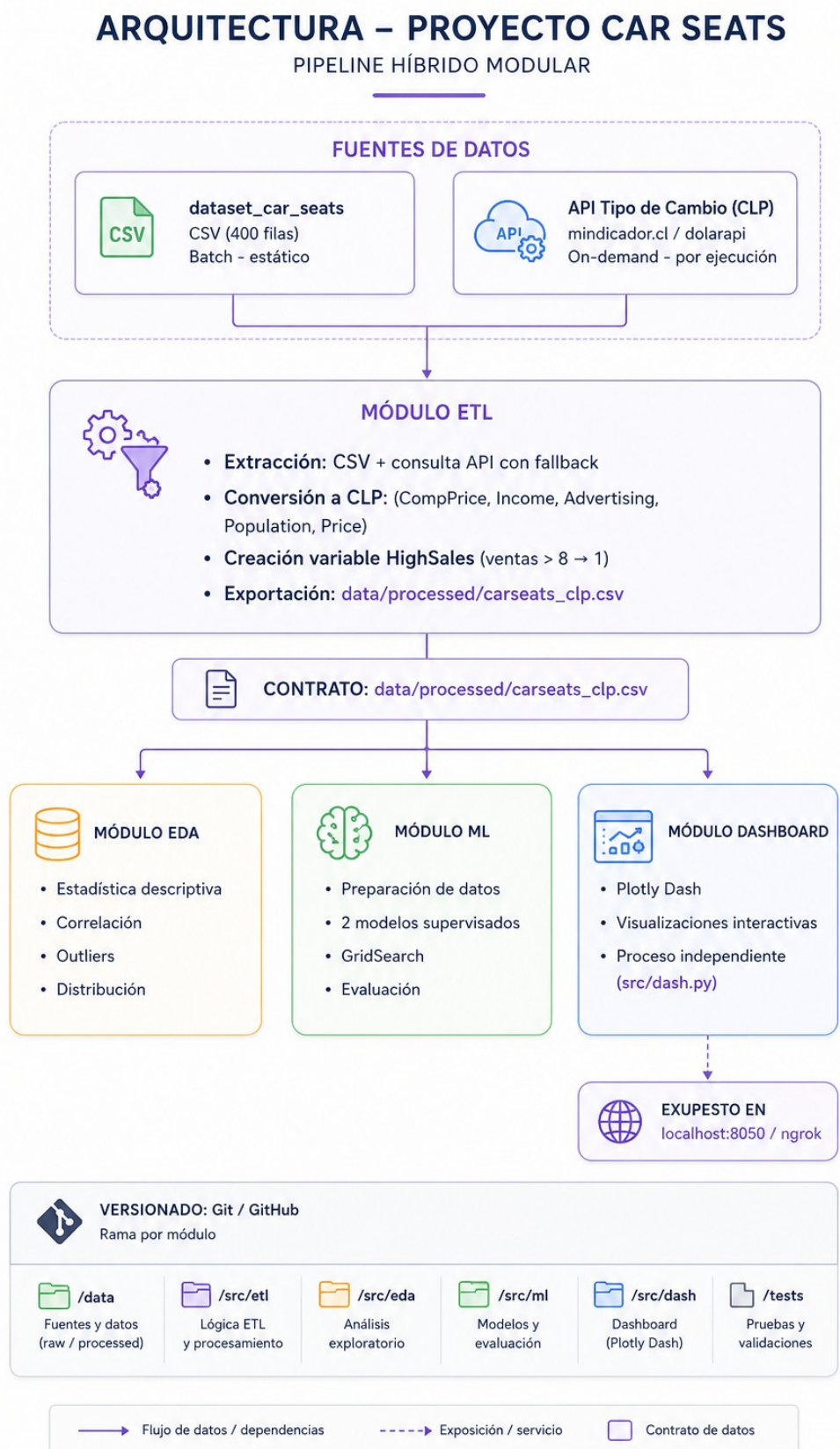
El Pipeline Híbrido Modular es la única de las tres que coincide con la naturaleza real del proyecto, debido a que combina una fuente batch (el CSV) con una fuente on-demand (la API), y organiza el resto del flujo en módulos independientes que se comunican a través de un archivo intermedio (data/processed/carseats_clp.csv).

El módulo de ETL no necesita saber qué hacen después el EDA, el modelado o el dashboard con ese archivo, y cada uno puede ejecutarse, modificarse o fallar de forma aislada sin afectar a los demás. Esta misma modularidad (si bien implementada en un solo archivo, está distribuido en secciones modulares) es la que permitió separar el dashboard en un proceso independiente sin que su naturaleza de servidor de larga duración bloqueara la ejecución del resto del notebook.

Característica del proyecto	Cómo la resuelve el Pipeline Híbrido Modular
CSV estático (400 filas)	Módulo de extracción batch simple: <code>pd.read_csv()</code>
API de tipo de cambio (on-demand)	Módulo de extracción on-demand con fallback en cadena
Transformaciones (HighSales, CLP)	Módulo de transformación aislado y reutilizable
EDA, ML y Dashboard independientes	Módulos de consumo débilmente acoplados vía archivo CSV
Dashboard como servidor de larga duración	Proceso separado (<code>src/dashboard.py</code>) sin bloquear el notebook
Versionado por Git	Rama de Git por módulo, integración limpia

5. Diagrama de la Arquitectura de la Solución

El diagrama siguiente representa el Pipeline Híbrido Modular implementado con las dos fuentes de datos alimentan el módulo de ETL, que exporta un dataset procesado único. Ese archivo actúa como contrato entre los tres módulos de consumo (EDA, modelado y dashboard), cada uno independiente entre sí.



5.1 Descripción del flujo de datos

Extracción (E)

El módulo de ETL extrae datos desde dos fuentes con naturalezas distintas, el archivo CSV (fuente batch, lectura única al inicio) y la API de tipo de cambio (fuente on-demand, consultada una vez por ejecución con fallback en cadena si la primera URL falla).

Transformación (T)

Se aplican las transformaciones requeridas como la conversión de columnas monetarias a pesos chilenos (CLP) usando el tipo de cambio obtenido de la API, y creación de la variable booleana HighSales que toma valor 1 si las ventas superan las 8 mil unidades y 0 en caso contrario.

Carga (L) y Contrato

El dataset transformado se exporta a data/processed/carseats_clp.csv. Este archivo actúa como el contrato del pipeline porque cualquier módulo posterior lo lee directamente sin necesidad de conocer cómo fue generado. Esto es lo que garantiza el desacoplamiento modular.

Consumo

Los tres módulos de consumo (EDA, ML y Dashboard) leen el mismo archivo procesado de forma independiente. Si se modifica el módulo de EDA, el de ML no se ve afectado. El dashboard corre como servidor de larga duración en un proceso separado para no bloquear la ejecución del notebook.

6. Conclusión

Lambda y Lakehouse son arquitecturas potentes, pero diseñadas para problemas de escala y tiempo real que este proyecto no tiene. El Pipeline Híbrido Modular, en cambio, ofrece exactamente lo que se necesitaba, que era una forma simple y desacoplada de combinar una fuente batch con una fuente on-demand, y de mantener independientes los módulos de análisis, modelado y visualización.

Elegir una arquitectura más compleja que la requerida no habría aportado valor real, solo costo y complejidad innecesaria. La arquitectura correcta es la que resuelve el problema con la menor complejidad suficiente, no la más sofisticada disponible.

Referencias

- Databricks. (2024). What is a Data Lakehouse? <https://www.databricks.com/glossary/data-lakehouse>
- Reis, J. & Housley, M. (2022). Fundamentals of Data Engineering. O'Reilly Media.
- O'Reilly. (2023). Data Architecture Patterns: Lambda, Kappa, and Beyond. <https://www.oreilly.com/library/view/data-architecture-patterns>